MAIE 5532: Machine Learning Systems

Week 7 - Machine Learning for Systems

Program: MSc in AI & Entrepreneurship

Term: Fall 2026/27

Instructor: Lefteris NTAFLOS, PhD

The Hong Kong University of Science and Technology

Machine Learning for Systems: Adaptive Optimization and System Intelligence

Learning Objectives:

- Master ML-driven system optimization across deployment scales
- Design adaptive systems with runtime optimization capabilities
- Implement energy-aware ML systems for embedded deployment
- Analyze automated optimization strategies and their trade-offs

Key Topics:

- ML-driven performance optimization
- Adaptive runtime systems
- Energy-aware optimization strategies
- Cross-scale system intelligence

Required Readings:

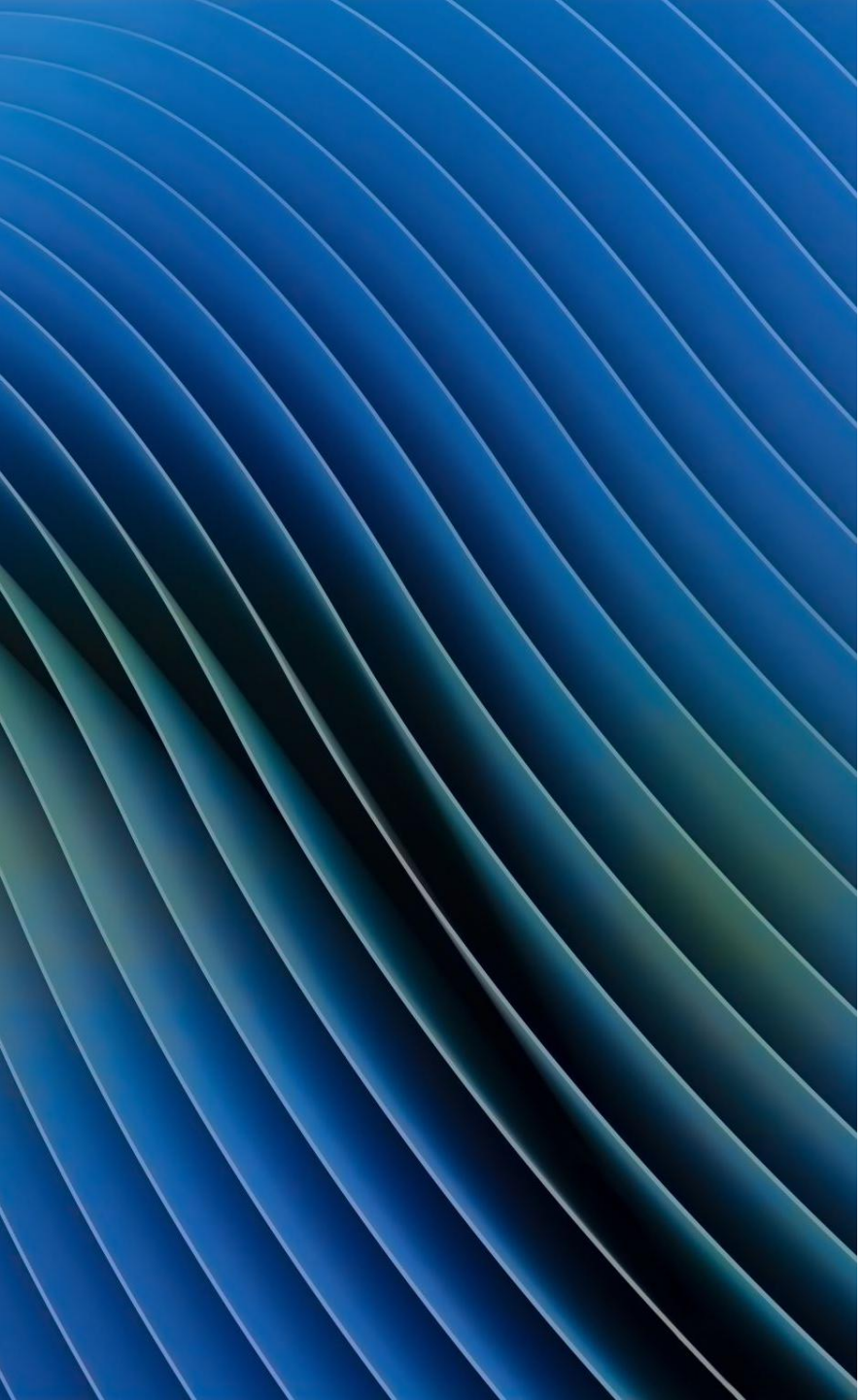
- Book 1 - Volume 2 — Performance Engineering
- Book 1 - Volume 2 — Edge Intelligence
- Book 1 - Volume 2 — Hardware Acceleration
- Book 2: Chapter 15: Optimizing Latency
- Book 2: Chapter 16: Optimizing Energy Usage



The Systems Intelligence Challenge



- **Traditional Approach Limitations**
  - Static heuristics and manual tuning
  - Fixed clock frequencies regardless of workload
  - Predetermined memory allocation strategies
  - Static operation scheduling
  - Based on worst-case assumptions
- **The ML Opportunity**
  - Dynamic optimization based on runtime characteristics
  - Adaptation to changing conditions
  - Decisions exceeding human-designed heuristics
  - Learning from actual usage patterns
- **Real-world Impact:** An edge device running object detection can learn optimal power management strategies, adapting clock frequencies and sleep patterns based on actual usage patterns rather than worst-case assumptions.



Session Structure

Session 1: ML-Driven System Optimization
- Performance prediction and modeling
- Automated parameter tuning
- Resource allocation optimization
- Cross-platform optimization strategies

Session 2: Adaptive Runtime Systems
- Dynamic optimization frameworks
- Runtime workload characterization
- Adaptive scheduling algorithms
- Self-tuning system architectures

Session 3: Energy-Aware ML Systems
- Power-performance optimization
- Duty cycling and frequency scaling
- Cascading system designs
- Battery lifetime optimization

SESSION 1: ML-DRIVEN SYSTEM OPTIMIZATION



Foundation Concepts

Core Principle: Moving from reactive to proactive system management through machine learning-driven optimization that learns from actual runtime behavior rather than relying on predetermined assumptions.

Traditional Optimization Challenges

- Static configuration for dynamic workloads
- Conservative performance estimates
- Manual parameter tuning complexity
- Limited adaptation to environmental changes

ML-Driven Solutions

- Adaptive configuration based on learned patterns
- Accurate performance prediction
- Automated optimization space exploration
- Environmental condition responsiveness

Performance Prediction Models

Learning System Behavior Patterns

The Prediction Challenge: Traditional static approaches estimate performance using simple formulas like operations per second times clock frequency. ML-enhanced approaches learn from actual execution patterns, capturing complex interactions between model characteristics, system state, and environmental conditions.

Feature Engineering for Performance

- **Model Characteristics:** Operation types, data access patterns, computational intensity
- **System State:** Memory pressure, thermal conditions, current utilization
- **Environmental Factors:** Ambient temperature, power availability, thermal constraints

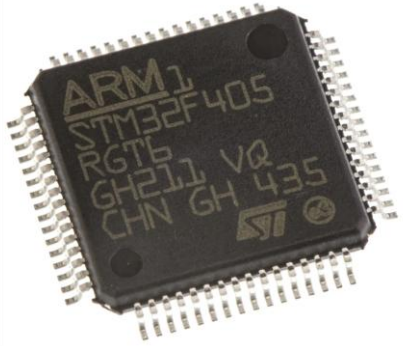
Benefits

- Accurate runtime estimation
- Proactive resource allocation
- Intelligent caching decisions

Applications

- Dynamic scheduling
- Memory management
- Thermal management

TinyML Performance Estimation



Model Latency Prediction for Embedded Systems

FLOP-Based Estimation Fundamentals: Most neural network models spend the majority of their time running large matrix multiplications. The work involved is approximately the number of input values times the number of output values for each layer.

Floating-Point Operations (FLOPs) as Performance Metric

- Usually a multiply-add operation counts as two FLOPs
- Even 8-bit quantized calculations are sometimes referred to as FLOPs
- Useful as rough metric for execution time comparison
- Model requiring 100M FLOPs runs ~2x faster than 200M FLOP version

Practical Example

- Model: 60M operations (person detector)
- Chip: Arm Cortex-M4 at 48 MHz
- Capability: 2 multiply/adds per cycle
- Estimated latency: 625 ms
- Power usage: ~1.25 mJ per inference

Limitations

- Doesn't account for software optimization
- Ignores memory access patterns
- Layer-specific optimizations vary
- Hardware features affect actual performance

Automated Parameter Tuning

ML-Guided Optimization Decisions

Optimization Space Complexity: Even for a single layer, there are thousands of ways to order computation loops, hundreds of parallelization strategies, and exponentially growing memory placement choices. This combinatorial explosion makes exhaustive search impractical.

Key Parameters to Optimize

- **Tile Size:** 16, 32, 64, 128 for memory optimization
- **Parallel Threads:** 1, 2, 4, 8 for computational parallelism
- **Memory Layout:** NHWC vs NCHW for cache efficiency
- **Kernel Fusion:** Enable/disable for reducing memory writes

Search Strategy Evolution

- Exhaustive search → impractical
- Random search → inefficient
- Reinforcement learning → learns patterns
- Bayesian optimization → efficient exploration

Learning Benefits

- Discovers non-obvious configurations
- Adapts to hardware characteristics
- Transfers knowledge across models
- Reduces optimization time

Mapping Strategies Building Blocks

Core Optimization Techniques

Structured Mapping Foundation: AI accelerators rely on structured mapping strategies that systematically balance computational efficiency, data locality, and parallel execution. Rather than evaluating every possible configuration, these approaches use heuristic, analytical, and ML-based techniques.

Data Movement Strategies

- Weight stationary approaches
- Output stationary methods
- Input stationary techniques
- Minimize redundant transfers

Memory-Aware Layouts

- Row-major organization
- Channel-major formatting
- Cache efficiency optimization
- Memory access pattern optimization

Advanced Techniques

- **Kernel Fusion:** Combines operations to minimize memory writes
- **Tiling:** Partitions large computations into memory-friendly blocks
- **Load Balancing:** Manages computation and communication trade-offs

Data Movement Patterns

Understanding Memory Access Challenges

Memory Access Bottleneck: Even when computational units are mapped efficiently, poor data movement strategies can severely degrade performance, leading to frequent memory stalls and underutilized hardware resources.

Matrix Multiplication Challenges

- **Memory Access Volume:** Each output requires full row and column fetches
- **Weight Reuse:** Same weights applied to multiple inputs
- **Intermediate Results:** Partial sums must be stored and retrieved
- **Bandwidth Demands:** Frequent memory access increases traffic

Impact on Performance

- Computational units remain idle
- Increased latency
- Higher memory traffic
- Elevated energy consumption

Solution Approach

- Dataflow strategies
- Maximize data reuse
- Reduce redundant fetches
- Optimize memory placement

Weight Stationary Strategy

Maximizing Weight Reuse

Strategy Overview: Weight Stationary keeps weights fixed in local memory while input activations and partial sums stream through the system. This approach is particularly beneficial in CNNs and matrix multiplications where the same weights are applied across multiple inputs.

Key Advantages

- Maximizes weight reuse
- Reduces redundant memory fetches
- Alleviates bandwidth bottlenecks
- Improves energy efficiency
- Effective for systolic arrays

Implementation Characteristics

- Weights loaded once into local memory
- Inputs streamed dynamically
- Partial sums accumulated efficiently
- High throughput execution
- Reduced external memory bandwidth

Trade-offs: While reducing weight-related memory traffic, this strategy introduces trade-offs in input and output movement. Efficiency depends on input delivery capabilities and partial sum accumulation strategies.

Output Stationary Strategy

Optimizing Accumulation Efficiency

Strategy Overview: Output Stationary keeps partial sums fixed in local memory while weights and input activations stream through the system. This approach is particularly effective for fully connected layers and operations where outputs accumulate contributions from multiple weight-input pairs.

Core Benefits

- Optimizes accumulation efficiency
- Reduces redundant memory writes
- Minimizes bandwidth consumption
- Improves energy efficiency
- Natural fit for systolic arrays

Best Applications

- Fully connected layers
- Attention mechanisms
- Matrix multiplications
- Transformer-based models
- Operations with heavy accumulation

Implementation Principles

- Partial sums remain in local memory throughout computation
- Weights and inputs stream dynamically
- Final outputs written only once to memory
- Eliminates excessive memory write traffic

Input Stationary Strategy

Maximizing Input Activation Reuse

Strategy Overview: Input Stationary keeps input activations fixed in local memory while weights and partial sums stream through the system. This approach is particularly effective for batch processing, transformer models, and sequence-based architectures.

Primary Advantages

- Maximizes input reuse
- Reduces memory access frequency
- Improves data locality
- Minimizes memory traffic
- Lowers energy consumption

Optimal Use Cases

- Transformer architectures
- Natural language processing
- Recommendation systems
- Large batch processing
- Sequence-based models

Implementation Benefits: Input activations are loaded into local memory and remain fixed during computation, while weights are streamed dynamically. This is particularly beneficial when each token in an input sequence is used across multiple attention heads and layers.

Memory-Aware Tensor Layouts



Optimizing Data Organization

Layout Importance: Tensor layouts significantly impact memory access efficiency, cache performance, and computational throughput. Poorly chosen layouts can lead to excessive memory stalls, inefficient cache usage, and increased data movement costs.

Layout Decision Makers

- **Framework Level:** TensorFlow, PyTorch, JAX automatic optimization
- **Compiler Level:** XLA, MLIR dynamic rearrangement
- **Hardware Libraries:** cuDNN, OneDNN layout enforcement
- **Developer Level:** Manual specification for custom kernels

Automatic Optimization

- Transparent framework transformations
- Hardware-aware layout selection
- Runtime optimization
- Target-specific adaptation

Manual Control

- Custom kernel development
- Low-level library usage
- Performance-critical applications
- Specialized hardware targets

Row-Major Layout (NHWC)

Sequential Memory Access Optimization

Layout Characteristics: Row-major layout stores multi-dimensional tensors row by row, ensuring all values in a given row are placed contiguously before moving to the next row. This aligns naturally with sequential memory access patterns.

Memory Storage Pattern

For a 3×3 RGB image tensor (Height, Width, Channels), values are stored as:
I(0,0,0), I(0,0,1), I(0,0,2), I(0,1,0), I(0,1,1), I(0,1,2)...
Each row is stored contiguously before moving to the next row.

Advantages

- Cache-friendly sequential access
- Optimal for CPU computation
- Efficient cache prefetching
- Good for element-wise operations
- Natural for batch normalization

Limitations

- Inefficient for channel-wise operations
- Strided memory access for convolutions
- Poor GPU memory coalescing
- Suboptimal for vectorized execution

Channel-Major Layout (NCHW)

Vectorized Computation Optimization

Layout Characteristics: Channel-major layout arranges data so all values for a given channel are stored together before moving to the next channel. This format is particularly beneficial for GPUs, TPUs, and AI accelerators.

Memory Storage Pattern

For a 3×3 RGB image, data is structured channel-first:

I(0,0,0), I(1,0,0), I(2,0,0), I(0,1,0)... [Red], I(0,0,1), I(1,0,1)... [Green], I(0,0,2)... [Blue]

All red channel values stored first, then green, then blue.

GPU/TPU Benefits

- Efficient memory coalescing
- Optimal for convolution operations
- SIMD execution alignment
- Parallel channel processing
- Reduced scattered memory fetches

Hardware Alignment

- GPU warp execution patterns
- TPU systolic array processing
- Contiguous thread memory access
- Vectorized instruction utilization

Row-Major vs Channel-Major Comparison

Performance Trade-offs and Hardware Compatibility

Feature	Row-Major (NHWC)	Channel-Major (NCHW)
Memory Storage Order	Pixels row-by-row, channels interleaved	All channel values stored together first
Best Hardware	CPUs, element-wise operations	GPUs, TPUs, convolution operations
Cache Efficiency	High locality for sequential row access	Optimized for memory coalescing across channels
Convolution Performance	Requires strided memory accesses (inefficient on GPUs)	Efficient for GPU convolution kernels
Memory Fetching	Good for row-sequential operations	Optimized for SIMD execution across channels
Framework Defaults	Default on CPUs (TensorFlow NHWC)	Default on GPUs (cuDNN prefers NCHW)

Dynamic Optimization: Modern AI compilers like XLA and TorchScript perform automatic layout transformations, converting between NHWC and NCHW as needed to optimize performance across different processing units.

Resource Allocation Intelligence

Dynamic Resource Management

Intelligent Memory Management: Systems learn optimal memory allocation strategies based on operation types, predicted usage patterns, and current memory state. This goes beyond simple best-fit algorithms to consider future memory requirements and fragmentation risks.

Memory Optimization Features

- Usage pattern prediction
- Fragmentation risk assessment
- Allocation history analysis
- Contiguous block management
- Cache-aware placement

Compute Resource Scaling

- Optimal processor frequency selection
- Core allocation based on workload
- Performance requirement balancing
- Energy consumption optimization
- Thermal constraint consideration

Learning Integration

- Systems continuously learn from allocation decisions and their outcomes, building models that predict optimal resource allocation strategies for different workload characteristics and system conditions.

Cross-Platform Optimization

Learning Across Hardware Targets

Knowledge Transfer Challenge: Optimization strategies that work well on one hardware platform may not transfer directly to another. However, many optimization principles can be adapted through intelligent transfer learning approaches.

Platform-Specific Models

- Hardware-specific optimization strategies
- Learned performance characteristics
- Platform-optimized configurations
- Validated optimization patterns

Transfer Learning Benefits

- Rapid optimization for new platforms
- Similarity-based adaptation
- Reduced optimization time
- Cross-platform pattern recognition

Implementation Strategy

Systems maintain platform-specific optimization models while learning similarities between platforms. When targeting a new platform, the system identifies the most similar existing platform and adapts successful optimization strategies.

Kernel Fusion for Memory Efficiency

Eliminating Intermediate Memory Writes

The Intermediate Memory Problem: AI models rely on high-throughput computation, but performance is often constrained by memory bandwidth and intermediate memory writes rather than pure arithmetic operations. Every intermediate result creates memory traffic overhead.

Naive Execution Overhead

- In traditional kernel-by-kernel execution, each operation treats intermediate results as separate tensors:X_1 = ReLU(X) → intermediate tensor stored in memory
- X_2 = BatchNorm(X_1) → another intermediate tensor stored
- Y = 2.0 * X_2 + 1.0 → final result
- This creates unnecessary memory traffic that can outweigh computational costs.

Memory Impact

- Multiple intermediate tensors
- 4x memory overhead for simple sequence
- Bandwidth consumption
- Cache pollution

Fusion Solution

- Y = a × BatchNorm(ReLU(X)) + b
- Single computational step
- No intermediate storage
- 75% memory reduction

Kernel Fusion Performance Benefits

Key Advantages

- **Reduced Memory Traffic:** Intermediate values stay in registers
- **Better Cache Utilization:** Eliminates redundant memory writes
- **Higher Throughput:** More compute unit utilization
- **Energy Efficiency:** Lower memory access energy cost

Fusion Limitations

- **Register Pressure:** Limited temporary value storage
- **Operation Compatibility:** Not all operations can fuse
- **Complex Dependencies:** Global data movement prevents fusion
- **Hardware Constraints:** Platform-specific limitations

Ideal Fusion Candidates: Element-wise operations such as ReLU, batch normalization, and arithmetic transformations are perfect for fusion since computations depend only on single input elements. Matrix multiplications and convolutions involve global data movement, making direct fusion impractical.

Platform Considerations: Different AI accelerators handle fusion differently. NVIDIA GPUs favor warp-level parallelism for element-wise fusion, while TPUs prioritize systolic array execution optimized for matrix operations.

Tiling for Memory Efficiency

Managing Memory Bandwidth Constraints

Memory Bandwidth Bottleneck: Modern AI accelerators offer high computational throughput, but performance is often limited by memory bandwidth rather than raw processing power. If data cannot be supplied fast enough to processing units, execution stalls occur.

Tiling Solution Principle: Tiling restructures computations into smaller, memory-friendly subproblems. Instead of processing entire matrices or tensors at once, computations are partitioned into smaller blocks (tiles) that fit within fast local memory.

Matrix Multiplication Problem

- Repeated memory accesses for same data
- Unnecessary bandwidth consumption
- DRAM access hundreds of times slower
- Performance dominated by memory latency

Tiling Benefits

- Better memory reuse
- Reduced bandwidth requirements
- Improved cache utilization
- Higher compute efficiency

Tiling Performance Improvement

Structured Memory Access Patterns

Tiling Implementation Strategy: Instead of computing one element at a time and constantly moving data between slow and fast memory, tiling processes submatrices (tiles) at a time, keeping frequently used values in fast memory throughout the computation.

Performance Improvement Mechanisms

- **Better Memory Reuse:** Small tiles loaded into fast memory, used multiple times
- **Reduced Memory Bandwidth:** Most data available in L1/L2 cache vs DRAM
- **Increased Compute Efficiency:** Processors spend less time waiting for data

AI Accelerator Benefits

- Thousands of parallel processing units
- Structured data reading and processing
- Avoidance of unnecessary stalls
- Optimal resource utilization

Application Areas

- Large matrix multiplications
- Tensor transformations
- Convolutional operations
- Attention mechanisms

Spatial vs Temporal Tiling

Spatial Tiling

Focus: Partitioning data structures into smaller blocks that fit within fast memory

- Each tile fully processed before moving to next
- Reduces redundant memory accesses
- Optimal for matrix multiplications
- Benefits convolutions and attention mechanisms
- Improves cache efficiency

Temporal Tiling (Loop Blocking)

Focus: Reorganizing computation to improve data reuse over time

- Same data accessed repeatedly across iterations
- Keeps frequently used data in fast memory longer
- Beneficial for convolutional operations
- Reduces weight loading overhead
- Optimal for recurrent computations

Hybrid Approaches

- Advanced systems combine spatial and temporal tiling, adapting dynamically to workload-specific constraints by adjusting tile sizes or reordering computations based on real-time execution conditions.

Quiz 1

Question 1

1.Which feature best captures the computational load relative to processing speed?

A) available_memory
B) ops_per_mhz
C) thermal_factor
D) power_constraint

Scenario
You are designing an intelligent performance prediction system for edge devices to optimize neural network inference. The system consumes two inputs:

1. System state information:
- cpu_frequency (Hz)
- available_memory (bytes)
- temperature (°C)
- battery_level (0–1)
- recent_inference_times (list of ms)

2. Model characteristics:
- total_operations (ops)
- peak_memory_usage (bytes)
- layer_types (e.g., conv2d, dense, relu)
- quantization (e.g., int8 or fp32)

Your feature-engineering solution computes:
- ops_per_mhz = total_operations / cpu_frequency
- memory_pressure = peak_memory_usage / available_memory
- thermal_factor = max(0, temperature − 25) / 25
- power_constraint = 1.0 − battery_level
- recent_trend = compute_trend(recent_inference_times)
- quantization_speedup = 1.0 if quantization == "int8" else 0.5
- layer_efficiency = compute_layer_efficiency(layer_types)

Predictions drive actions like dynamic frequency scaling, intelligent batching, and proactive thermal management.

Quiz 1

Answer 1

1.Which feature best captures the computational load relative to processing speed?

A) available_memory
B) ops_per_mhz
C) thermal_factor
D) power_constraint

ops_per_mhz relates total operations to CPU frequency, normalizing workload by compute speed.

Scenario
You are designing an intelligent performance prediction system for edge devices to optimize neural network inference. The system consumes two inputs:

1. System state information:
- cpu_frequency (Hz)
- available_memory (bytes)
- temperature (°C)
- battery_level (0–1)
- recent_inference_times (list of ms)

2. Model characteristics:
- total_operations (ops)
- peak_memory_usage (bytes)
- layer_types (e.g., conv2d, dense, relu)
- quantization (e.g., int8 or fp32)

Your feature-engineering solution computes:
- ops_per_mhz = total_operations / cpu_frequency
- memory_pressure = peak_memory_usage / available_memory
- thermal_factor = max(0, temperature − 25) / 25
- power_constraint = 1.0 − battery_level
- recent_trend = compute_trend(recent_inference_times)
- quantization_speedup = 1.0 if quantization == "int8" else 0.5
- layer_efficiency = compute_layer_efficiency(layer_types)

Predictions drive actions like dynamic frequency scaling, intelligent batching, and proactive thermal management.

Quiz 1

Question 2

What does memory_pressure primarily indicate?

A) Probability of thermal throttling
B) The need to quantize the model
C) Risk of paging/OOM due to model footprint vs memory
D) Trend in latency over time

Scenario
You are designing an intelligent performance prediction system for edge devices to optimize neural network inference. The system consumes two inputs:

1. System state information:
- cpu_frequency (Hz)
- available_memory (bytes)
- temperature (°C)
- battery_level (0–1)
- recent_inference_times (list of ms)

2. Model characteristics:
- total_operations (ops)
- peak_memory_usage (bytes)
- layer_types (e.g., conv2d, dense, relu)
- quantization (e.g., int8 or fp32)

Your feature-engineering solution computes:
- ops_per_mhz = total_operations / cpu_frequency
- memory_pressure = peak_memory_usage / available_memory
- thermal_factor = max(0, temperature − 25) / 25
- power_constraint = 1.0 − battery_level
- recent_trend = compute_trend(recent_inference_times)
- quantization_speedup = 1.0 if quantization == "int8" else 0.5
- layer_efficiency = compute_layer_efficiency(layer_types)

Predictions drive actions like dynamic frequency scaling, intelligent batching, and proactive thermal management.

Quiz 1

Answer 2

What does memory_pressure primarily indicate?

A) Probability of thermal throttling
B) The need to quantize the model
C) Risk of paging/OOM due to model footprint vs memory
D) Trend in latency over time

memory_pressure = peak_memory_usage / available_memory estimates how close the model comes to exhausting RAM.

Scenario
You are designing an intelligent performance prediction system for edge devices to optimize neural network inference. The system consumes two inputs:

1. System state information:
- cpu_frequency (Hz)
- available_memory (bytes)
- temperature (°C)
- battery_level (0–1)
- recent_inference_times (list of ms)

2. Model characteristics:
- total_operations (ops)
- peak_memory_usage (bytes)
- layer_types (e.g., conv2d, dense, relu)
- quantization (e.g., int8 or fp32)

Your feature-engineering solution computes:
- ops_per_mhz = total_operations / cpu_frequency
- memory_pressure = peak_memory_usage / available_memory
- thermal_factor = max(0, temperature − 25) / 25
- power_constraint = 1.0 − battery_level
- recent_trend = compute_trend(recent_inference_times)
- quantization_speedup = 1.0 if quantization == "int8" else 0.5
- layer_efficiency = compute_layer_efficiency(layer_types)

Predictions drive actions like dynamic frequency scaling, intelligent batching, and proactive thermal management.

Quiz 1

Question 3

Why is thermal_factor computed as max(0, temperature − 25) / 25?

A) To penalize low temperatures
B) To bound the value between −1 and 1
C) To represent throttling risk only when above a baseline
D) To convert Celsius to Fahrenheit

Scenario
You are designing an intelligent performance prediction system for edge devices to optimize neural network inference. The system consumes two inputs:

1. System state information:
- cpu_frequency (Hz)
- available_memory (bytes)
- temperature (°C)
- battery_level (0–1)
- recent_inference_times (list of ms)

2. Model characteristics:
- total_operations (ops)
- peak_memory_usage (bytes)
- layer_types (e.g., conv2d, dense, relu)
- quantization (e.g., int8 or fp32)

Your feature-engineering solution computes:
- ops_per_mhz = total_operations / cpu_frequency
- memory_pressure = peak_memory_usage / available_memory
- thermal_factor = max(0, temperature − 25) / 25
- power_constraint = 1.0 − battery_level
- recent_trend = compute_trend(recent_inference_times)
- quantization_speedup = 1.0 if quantization == "int8" else 0.5
- layer_efficiency = compute_layer_efficiency(layer_types)

Predictions drive actions like dynamic frequency scaling, intelligent batching, and proactive thermal management.

Quiz 1

Answer 3

Why is thermal_factor computed as max(0, temperature − 25) / 25?

A) To penalize low temperatures
B) To bound the value between −1 and 1
C) To represent throttling risk only when above a baseline
D) To convert Celsius to Fahrenheit

Using max with a baseline ignores safe temps and scales risk as temperature rises over an assumed safe baseline (~25°C).

Scenario
You are designing an intelligent performance prediction system for edge devices to optimize neural network inference. The system consumes two inputs:

1. System state information:
- cpu_frequency (Hz)
- available_memory (bytes)
- temperature (°C)
- battery_level (0–1)
- recent_inference_times (list of ms)

2. Model characteristics:
- total_operations (ops)
- peak_memory_usage (bytes)
- layer_types (e.g., conv2d, dense, relu)
- quantization (e.g., int8 or fp32)

Your feature-engineering solution computes:
- ops_per_mhz = total_operations / cpu_frequency
- memory_pressure = peak_memory_usage / available_memory
- thermal_factor = max(0, temperature − 25) / 25
- power_constraint = 1.0 − battery_level
- recent_trend = compute_trend(recent_inference_times)
- quantization_speedup = 1.0 if quantization == "int8" else 0.5
- layer_efficiency = compute_layer_efficiency(layer_types)

Predictions drive actions like dynamic frequency scaling, intelligent batching, and proactive thermal management.

Quiz 1

Question 4

power_constraint = 1.0 – battery_level captures:

A) CPU utilization
B) Urgency to save power when battery is low
C) GPU memory bandwidth
D) Radio signal quality

Scenario
You are designing an intelligent performance prediction system for edge devices to optimize neural network inference. The system consumes two inputs:

1. System state information:
- cpu_frequency (Hz)
- available_memory (bytes)
- temperature (°C)
- battery_level (0–1)
- recent_inference_times (list of ms)

2. Model characteristics:
- total_operations (ops)
- peak_memory_usage (bytes)
- layer_types (e.g., conv2d, dense, relu)
- quantization (e.g., int8 or fp32)

Your feature-engineering solution computes:
- ops_per_mhz = total_operations / cpu_frequency
- memory_pressure = peak_memory_usage / available_memory
- thermal_factor = max(0, temperature − 25) / 25
- power_constraint = 1.0 − battery_level
- recent_trend = compute_trend(recent_inference_times)
- quantization_speedup = 1.0 if quantization == "int8" else 0.5
- layer_efficiency = compute_layer_efficiency(layer_types)

Predictions drive actions like dynamic frequency scaling, intelligent batching, and proactive thermal management.

Quiz 1

Answer 4

power_constraint = 1.0 – battery_level captures:

A) CPU utilization
B) Urgency to save power when battery is low
C) GPU memory bandwidth
D) Radio signal quality

As battery_level drops, power_constraint increases, signaling stronger pressure to conserve energy and reduce latency/compute.

Scenario
You are designing an intelligent performance prediction system for edge devices to optimize neural network inference. The system consumes two inputs:

1. System state information:
- cpu_frequency (Hz)
- available_memory (bytes)
- temperature (°C)
- battery_level (0–1)
- recent_inference_times (list of ms)

2. Model characteristics:
- total_operations (ops)
- peak_memory_usage (bytes)
- layer_types (e.g., conv2d, dense, relu)
- quantization (e.g., int8 or fp32)

Your feature-engineering solution computes:
- ops_per_mhz = total_operations / cpu_frequency
- memory_pressure = peak_memory_usage / available_memory
- thermal_factor = max(0, temperature − 25) / 25
- power_constraint = 1.0 − battery_level
- recent_trend = compute_trend(recent_inference_times)
- quantization_speedup = 1.0 if quantization == "int8" else 0.5
- layer_efficiency = compute_layer_efficiency(layer_types)

Predictions drive actions like dynamic frequency scaling, intelligent batching, and proactive thermal management.

Quiz 1

Question 5

How does quantization_speedup influence predicted latency?

A) Increases latency for int8 models
B) Decreases latency by reflecting faster integer ops
C) Only affects memory, not speed
D) Removes the need for other features

Scenario
You are designing an intelligent performance prediction system for edge devices to optimize neural network inference. The system consumes two inputs:

1. System state information:
- cpu_frequency (Hz)
- available_memory (bytes)
- temperature (°C)
- battery_level (0–1)
- recent_inference_times (list of ms)

2. Model characteristics:
- total_operations (ops)
- peak_memory_usage (bytes)
- layer_types (e.g., conv2d, dense, relu)
- quantization (e.g., int8 or fp32)

Your feature-engineering solution computes:
- ops_per_mhz = total_operations / cpu_frequency
- memory_pressure = peak_memory_usage / available_memory
- thermal_factor = max(0, temperature − 25) / 25
- power_constraint = 1.0 − battery_level
- recent_trend = compute_trend(recent_inference_times)
- quantization_speedup = 1.0 if quantization == "int8" else 0.5
- layer_efficiency = compute_layer_efficiency(layer_types)

Predictions drive actions like dynamic frequency scaling, intelligent batching, and proactive thermal management.

Quiz 1

Answer 5

How does quantization_speedup influence predicted latency?

A) Increases latency for int8 models
B) Decreases latency by reflecting faster integer ops
C) Only affects memory, not speed
D) Removes the need for other features

Setting higher speedup for int8 encodes that quantized models typically run faster on edge hardware.

Scenario
You are designing an intelligent performance prediction system for edge devices to optimize neural network inference. The system consumes two inputs:

1. System state information:
- cpu_frequency (Hz)
- available_memory (bytes)
- temperature (°C)
- battery_level (0–1)
- recent_inference_times (list of ms)

2. Model characteristics:
- total_operations (ops)
- peak_memory_usage (bytes)
- layer_types (e.g., conv2d, dense, relu)
- quantization (e.g., int8 or fp32)

Your feature-engineering solution computes:
- ops_per_mhz = total_operations / cpu_frequency
- memory_pressure = peak_memory_usage / available_memory
- thermal_factor = max(0, temperature − 25) / 25
- power_constraint = 1.0 − battery_level
- recent_trend = compute_trend(recent_inference_times)
- quantization_speedup = 1.0 if quantization == "int8" else 0.5
- layer_efficiency = compute_layer_efficiency(layer_types)

Predictions drive actions like dynamic frequency scaling, intelligent batching, and proactive thermal management.

Quiz 1

Question 6

recent_trend from recent_inference_times is useful because it:

A) Replaces ops_per_mhz
B) Captures non-stationary effects like thermal drift and contention
C) Measures static model size
D) Ensures overfitting

Scenario
You are designing an intelligent performance prediction system for edge devices to optimize neural network inference. The system consumes two inputs:

1. System state information:
- cpu_frequency (Hz)
- available_memory (bytes)
- temperature (°C)
- battery_level (0–1)
- recent_inference_times (list of ms)

2. Model characteristics:
- total_operations (ops)
- peak_memory_usage (bytes)
- layer_types (e.g., conv2d, dense, relu)
- quantization (e.g., int8 or fp32)

Your feature-engineering solution computes:
- ops_per_mhz = total_operations / cpu_frequency
- memory_pressure = peak_memory_usage / available_memory
- thermal_factor = max(0, temperature − 25) / 25
- power_constraint = 1.0 − battery_level
- recent_trend = compute_trend(recent_inference_times)
- quantization_speedup = 1.0 if quantization == "int8" else 0.5
- layer_efficiency = compute_layer_efficiency(layer_types)

Predictions drive actions like dynamic frequency scaling, intelligent batching, and proactive thermal management.

Quiz 1

Answer 6

recent_trend from recent_inference_times is useful because it:

A) Replaces ops_per_mhz
B) Captures non-stationary effects like thermal drift and contention
C) Measures static model size
D) Ensures overfitting

Trend over time reflects evolving runtime conditions that static features miss, improving short-term predictions.

Scenario
You are designing an intelligent performance prediction system for edge devices to optimize neural network inference. The system consumes two inputs:

1. System state information:
- cpu_frequency (Hz)
- available_memory (bytes)
- temperature (°C)
- battery_level (0–1)
- recent_inference_times (list of ms)

2. Model characteristics:
- total_operations (ops)
- peak_memory_usage (bytes)
- layer_types (e.g., conv2d, dense, relu)
- quantization (e.g., int8 or fp32)

Your feature-engineering solution computes:
- ops_per_mhz = total_operations / cpu_frequency
- memory_pressure = peak_memory_usage / available_memory
- thermal_factor = max(0, temperature − 25) / 25
- power_constraint = 1.0 − battery_level
- recent_trend = compute_trend(recent_inference_times)
- quantization_speedup = 1.0 if quantization == "int8" else 0.5
- layer_efficiency = compute_layer_efficiency(layer_types)

Predictions drive actions like dynamic frequency scaling, intelligent batching, and proactive thermal management.

Quiz 1

Question 7

layer_efficiency computed from layer_types helps because:

A) All layers have identical throughput
B) It encodes operator-specific hardware performance
C) It duplicates total_operations
D) It measures battery level

Scenario
You are designing an intelligent performance prediction system for edge devices to optimize neural network inference. The system consumes two inputs:

1. System state information:
- cpu_frequency (Hz)
- available_memory (bytes)
- temperature (°C)
- battery_level (0–1)
- recent_inference_times (list of ms)

2. Model characteristics:
- total_operations (ops)
- peak_memory_usage (bytes)
- layer_types (e.g., conv2d, dense, relu)
- quantization (e.g., int8 or fp32)

Your feature-engineering solution computes:
- ops_per_mhz = total_operations / cpu_frequency
- memory_pressure = peak_memory_usage / available_memory
- thermal_factor = max(0, temperature − 25) / 25
- power_constraint = 1.0 − battery_level
- recent_trend = compute_trend(recent_inference_times)
- quantization_speedup = 1.0 if quantization == "int8" else 0.5
- layer_efficiency = compute_layer_efficiency(layer_types)

Predictions drive actions like dynamic frequency scaling, intelligent batching, and proactive thermal management.

Quiz 1

Answer 7

layer_efficiency computed from layer_types helps because:

A) All layers have identical throughput
B) It encodes operator-specific hardware performance
C) It duplicates total_operations
D) It measures battery level

Different layers map differently to hardware; encoding their mix captures efficiency differences.

Scenario
You are designing an intelligent performance prediction system for edge devices to optimize neural network inference. The system consumes two inputs:

1. System state information:
- cpu_frequency (Hz)
- available_memory (bytes)
- temperature (°C)
- battery_level (0–1)
- recent_inference_times (list of ms)

2. Model characteristics:
- total_operations (ops)
- peak_memory_usage (bytes)
- layer_types (e.g., conv2d, dense, relu)
- quantization (e.g., int8 or fp32)

Your feature-engineering solution computes:
- ops_per_mhz = total_operations / cpu_frequency
- memory_pressure = peak_memory_usage / available_memory
- thermal_factor = max(0, temperature − 25) / 25
- power_constraint = 1.0 − battery_level
- recent_trend = compute_trend(recent_inference_times)
- quantization_speedup = 1.0 if quantization == "int8" else 0.5
- layer_efficiency = compute_layer_efficiency(layer_types)

Predictions drive actions like dynamic frequency scaling, intelligent batching, and proactive thermal management.

Quiz 1

Question 8

Which action is most directly triggered by a high thermal_factor prediction?

A) Increase batch size
B) Raise CPU frequency
C) Disable quantization
D) Throttle frequency or schedule cool-down

Scenario
You are designing an intelligent performance prediction system for edge devices to optimize neural network inference. The system consumes two inputs:

1. System state information:
- cpu_frequency (Hz)
- available_memory (bytes)
- temperature (°C)
- battery_level (0–1)
- recent_inference_times (list of ms)

2. Model characteristics:
- total_operations (ops)
- peak_memory_usage (bytes)
- layer_types (e.g., conv2d, dense, relu)
- quantization (e.g., int8 or fp32)

Your feature-engineering solution computes:
- ops_per_mhz = total_operations / cpu_frequency
- memory_pressure = peak_memory_usage / available_memory
- thermal_factor = max(0, temperature − 25) / 25
- power_constraint = 1.0 − battery_level
- recent_trend = compute_trend(recent_inference_times)
- quantization_speedup = 1.0 if quantization == "int8" else 0.5
- layer_efficiency = compute_layer_efficiency(layer_types)

Predictions drive actions like dynamic frequency scaling, intelligent batching, and proactive thermal management.

Quiz 1

Answer 8

Which action is most directly triggered by a high thermal_factor prediction?

A) Increase batch size
B) Raise CPU frequency
C) Disable quantization
D) Throttle frequency or schedule cool-down

High thermal risk suggests reducing frequency or pausing to avoid throttling and stabilize latency.

Scenario
You are designing an intelligent performance prediction system for edge devices to optimize neural network inference. The system consumes two inputs:

1. System state information:
- cpu_frequency (Hz)
- available_memory (bytes)
- temperature (°C)
- battery_level (0–1)
- recent_inference_times (list of ms)

2. Model characteristics:
- total_operations (ops)
- peak_memory_usage (bytes)
- layer_types (e.g., conv2d, dense, relu)
- quantization (e.g., int8 or fp32)

Your feature-engineering solution computes:
- ops_per_mhz = total_operations / cpu_frequency
- memory_pressure = peak_memory_usage / available_memory
- thermal_factor = max(0, temperature − 25) / 25
- power_constraint = 1.0 − battery_level
- recent_trend = compute_trend(recent_inference_times)
- quantization_speedup = 1.0 if quantization == "int8" else 0.5
- layer_efficiency = compute_layer_efficiency(layer_types)

Predictions drive actions like dynamic frequency scaling, intelligent batching, and proactive thermal management.

Quiz 1

Question 9

If memory_pressure > 1.0, the most likely impact is:

A) Out-of-memory or heavy
B) Lower latency due to caching swapping causing large latency spikes
C) Improved thermal behavior
D) Higher battery level

Scenario
You are designing an intelligent performance prediction system for edge devices to optimize neural network inference. The system consumes two inputs:

1. System state information:
- cpu_frequency (Hz)
- available_memory (bytes)
- temperature (°C)
- battery_level (0–1)
- recent_inference_times (list of ms)

2. Model characteristics:
- total_operations (ops)
- peak_memory_usage (bytes)
- layer_types (e.g., conv2d, dense, relu)
- quantization (e.g., int8 or fp32)

Your feature-engineering solution computes:
- ops_per_mhz = total_operations / cpu_frequency
- memory_pressure = peak_memory_usage / available_memory
- thermal_factor = max(0, temperature − 25) / 25
- power_constraint = 1.0 − battery_level
- recent_trend = compute_trend(recent_inference_times)
- quantization_speedup = 1.0 if quantization == "int8" else 0.5
- layer_efficiency = compute_layer_efficiency(layer_types)

Predictions drive actions like dynamic frequency scaling, intelligent batching, and proactive thermal management.

Quiz 1

Answer 9

If memory_pressure > 1.0, the most likely impact is:

A) Out-of-memory or heavy
B) Lower latency due to caching swapping causing large latency spikes
C) Improved thermal behavior
D) Higher battery level

Exceeding available memory leads to OOM or paging, dramatically increasing latency.

Scenario
You are designing an intelligent performance prediction system for edge devices to optimize neural network inference. The system consumes two inputs:

1. System state information:
- cpu_frequency (Hz)
- available_memory (bytes)
- temperature (°C)
- battery_level (0–1)
- recent_inference_times (list of ms)

2. Model characteristics:
- total_operations (ops)
- peak_memory_usage (bytes)
- layer_types (e.g., conv2d, dense, relu)
- quantization (e.g., int8 or fp32)

Your feature-engineering solution computes:
- ops_per_mhz = total_operations / cpu_frequency
- memory_pressure = peak_memory_usage / available_memory
- thermal_factor = max(0, temperature − 25) / 25
- power_constraint = 1.0 − battery_level
- recent_trend = compute_trend(recent_inference_times)
- quantization_speedup = 1.0 if quantization == "int8" else 0.5
- layer_efficiency = compute_layer_efficiency(layer_types)

Predictions drive actions like dynamic frequency scaling, intelligent batching, and proactive thermal management.

Quiz 1

Question 10

For system-level optimization, combining predictions with batching is most beneficial when:

A) Latency target is loose and ops_per_mhz is low
B) Thermal_factor is high and battery low
C) Memory_pressure > 1.0
D) recent_trend shows increasing latency

Scenario
You are designing an intelligent performance prediction system for edge devices to optimize neural network inference. The system consumes two inputs:

1. System state information:
- cpu_frequency (Hz)
- available_memory (bytes)
- temperature (°C)
- battery_level (0–1)
- recent_inference_times (list of ms)

2. Model characteristics:
- total_operations (ops)
- peak_memory_usage (bytes)
- layer_types (e.g., conv2d, dense, relu)
- quantization (e.g., int8 or fp32)

Your feature-engineering solution computes:
- ops_per_mhz = total_operations / cpu_frequency
- memory_pressure = peak_memory_usage / available_memory
- thermal_factor = max(0, temperature − 25) / 25
- power_constraint = 1.0 − battery_level
- recent_trend = compute_trend(recent_inference_times)
- quantization_speedup = 1.0 if quantization == "int8" else 0.5
- layer_efficiency = compute_layer_efficiency(layer_types)

Predictions drive actions like dynamic frequency scaling, intelligent batching, and proactive thermal management.

Quiz 1

Answer 10

For system-level optimization, combining predictions with batching is most beneficial when:

A) Latency target is loose and ops_per_mhz is low
B) Thermal_factor is high and battery low
C) Memory_pressure > 1.0
D) recent_trend shows increasing latency

When compute headroom exists (low ops_per_mhz) and latency SLAs are relaxed, batching can improve throughput without violating constraints.

Scenario
You are designing an intelligent performance prediction system for edge devices to optimize neural network inference. The system consumes two inputs:

1. System state information:
- cpu_frequency (Hz)
- available_memory (bytes)
- temperature (°C)
- battery_level (0–1)
- recent_inference_times (list of ms)

2. Model characteristics:
- total_operations (ops)
- peak_memory_usage (bytes)
- layer_types (e.g., conv2d, dense, relu)
- quantization (e.g., int8 or fp32)

Your feature-engineering solution computes:
- ops_per_mhz = total_operations / cpu_frequency
- memory_pressure = peak_memory_usage / available_memory
- thermal_factor = max(0, temperature − 25) / 25
- power_constraint = 1.0 − battery_level
- recent_trend = compute_trend(recent_inference_times)
- quantization_speedup = 1.0 if quantization == "int8" else 0.5
- layer_efficiency = compute_layer_efficiency(layer_types)

Predictions drive actions like dynamic frequency scaling, intelligent batching, and proactive thermal management.

SESSION 2: ADAPTIVE RUNTIME SYSTEMS

Dynamic Optimization Foundations

- **Paradigm Shift**
- Adaptive runtime systems represent a fundamental shift from static system configuration to intelligent, responsive system management that improves performance through continuous learning and adaptation.

Core Requirements

- **Monitoring:** Capture relevant system behavior metrics
- **Decision Making:** Analyze data to identify optimization opportunities
- **Adaptation:** Apply configuration changes safely and efficiently
- **Learning:** Improve optimization strategies over time

Design Challenges

- Minimizing monitoring overhead
- Rapid decision-making requirements
- Safe adaptation mechanisms
- Coordinating multiple optimizations

Success Factors

- Efficient performance monitoring
- Intelligent adaptation strategies
- Stable system behavior
- Continuous improvement capability

Workload Characterization

Understanding System Behavior Patterns

Beyond Simple Metrics: Effective system optimization requires deep understanding of workload characteristics beyond operation counts. ML systems automatically characterize workloads based on memory access patterns, computational intensity, and parallelization opportunities.

Pattern Recognition Categories

- **Streaming Computation:** Continuous data processing with steady resource needs
- **Batch Processing:** Periodic high-intensity computation phases
- **Interactive Inference:** Sporadic requests with latency constraints
- **Periodic Processing:** Regular computation cycles with predictable patterns

Computational Intensity Classification

- **Compute-bound:** High arithmetic intensity
- **Memory-bound:** Limited by bandwidth
- **I/O-bound:** Constrained by data movement

Temporal Pattern Analysis

- Varying computational requirements
- Predictive optimization opportunities
- Proactive resource management



Intelligent Scheduling

ML-Driven Task Management

- **Beyond Traditional Scheduling**
- Traditional schedulers use simple policies like round-robin or priority-based scheduling. ML-enhanced schedulers learn optimal scheduling policies from execution traces and system feedback.

Multi-Objective Scheduling

- **Latency Minimization:** Reduce response times
- **Energy Efficiency:** Optimize power consumption
- **Thermal Management:** Prevent overheating
- **Fairness:** Ensure resource equity

Advanced Capabilities

- **Predictive Scheduling:** Anticipate task characteristics
- **System State Evolution:** Predict resource availability
- **Learning from Experience:** Improve over time
- **Adaptation:** Respond to changing patterns

Implementation Benefits: Intelligent schedulers enable automatic discovery of scheduling policies that balance competing objectives, improving overall system performance while adapting to changing workload patterns and system characteristics.

Dynamic Optimization Frameworks

Runtime Adaptation Mechanisms

Adaptation Architecture Requirements: Adaptive systems require sophisticated architectures that monitor system behavior, make optimization decisions, and apply adaptations without disrupting ongoing computation. The challenge lies in minimizing adaptation overhead while maximizing optimization benefits.

Policy-Based Adaptation

- Multiple optimization policies maintained
- Condition-specific policy selection
- ML-guided policy choice
- Current state and future prediction integration

Feedback Control Integration

- Control theory principles application
- Desired performance characteristic maintenance
- Oscillation and instability avoidance
- Automatic parameter adjustment

Coordination Challenges: Multiple adaptation mechanisms must coordinate their actions to avoid conflicts. A global coordination framework ensures that different optimizations work together rather than interfering with each other.

Runtime Model Selection

Intelligent Model Switching

Adaptive Model Management Philosophy: Systems maintain multiple model variants and intelligently switch between them based on current operating conditions and performance requirements, optimizing accuracy-performance trade-offs dynamically.

Model Variant Categories

- **High Accuracy:** Large model for optimal results when resources allow
- **Balanced:** Medium model for typical operating conditions
- **Low Latency:** Small model for resource-constrained scenarios

Selection Criteria

- Input complexity analysis
- Available resource assessment
- Performance requirement evaluation
- Energy constraint consideration

Implementation Benefits

- Complex models when resources abundant
- Simple models when constrained
- Continuous optimization
- Dynamic trade-off management

Self-Tuning Parameters

Automated Configuration Adaptation

Parameter Adaptation Framework: Self-tuning systems automatically adjust configuration parameters based on performance feedback and changing conditions, eliminating manual parameter tuning while maintaining optimal performance across varying operating conditions.

Control Theory Applications

- **PID Controllers:** Proportional-integral-derivative control
- **Advanced Control:** More sophisticated strategies
- **Performance Maintenance:** Desired characteristic preservation
- **Oscillation Prevention:** Parameter stability assurance

Multi-Scale Adaptation

- **Real-time:** Processor frequency (milliseconds)
- **Short-term:** Model selection (seconds)
- **Long-term:** Architecture changes (minutes)
- **Strategic:** System-wide optimizations (hours)

Stability Considerations: Self-tuning systems must maintain stability while exploring parameter spaces. Techniques include conservative adaptation, gradual parameter changes, and stability monitoring to prevent performance degradation.

Quiz 2

Question 1

1.What is the first decision layer in the hierarchical framework?

A) Select highest FPS possible
B) Maximize resolution
C) Check critical constraints (battery/thermal) before optimizing performance
D) Choose detection model with highest mAP

Scenario

You're designing an adaptive smart camera system that modifies its processing strategy based on environment and resource constraints. The camera supports multiple detection modes (motion, object, face), variable resolution and frame rate, dynamic power management, and adaptive quality settings. It must handle four adaptation scenarios:

1. High Activity Period: many objects, good lighting.
2. Battery Critical: low power mode required.
3. Thermal Throttling: device is hot.
4. Night Mode: low light; simplified detection.

Your solution framework emphasizes:

- Hierarchical decision making: prioritize hard constraints (battery, thermal) before performance.
- Adaptive thresholds: learn switching points from historical data.
- Gradual adaptation: smooth transitions to preserve UX.
- Learning integration: continuously improve adaptation policies.

Quiz 2

Answer 1

1.What is the first decision layer in the hierarchical framework?

A) Select highest FPS possible
B) Maximize resolution
C) Check critical constraints (battery/thermal) before optimizing performance
D) Choose detection model with highest mAP

The hierarchy enforces safety and device health before tuning for performance.

Scenario

You're designing an adaptive smart camera system that modifies its processing strategy based on environment and resource constraints. The camera supports multiple detection modes (motion, object, face), variable resolution and frame rate, dynamic power management, and adaptive quality settings. It must handle four adaptation scenarios:

1. High Activity Period: many objects, good lighting.
2. Battery Critical: low power mode required.
3. Thermal Throttling: device is hot.
4. Night Mode: low light; simplified detection.

Your solution framework emphasizes:

- Hierarchical decision making: prioritize hard constraints (battery, thermal) before performance.
- Adaptive thresholds: learn switching points from historical data.
- Gradual adaptation: smooth transitions to preserve UX.
- Learning integration: continuously improve adaptation policies.

Quiz 2

Question 2

In Battery Critical mode, which choice best aligns with the framework?

A) Increase frame rate to reduce motion blur
B) Switch to lightweight motion detection and lower resolution
C) Enable HDR and wide dynamic range
D) Run all detectors in parallel for redundancy

Scenario

You're designing an adaptive smart camera system that modifies its processing strategy based on environment and resource constraints. The camera supports multiple detection modes (motion, object, face), variable resolution and frame rate, dynamic power management, and adaptive quality settings. It must handle four adaptation scenarios:

1. High Activity Period: many objects, good lighting.
2. Battery Critical: low power mode required.
3. Thermal Throttling: device is hot.
4. Night Mode: low light; simplified detection.

Your solution framework emphasizes:

- Hierarchical decision making: prioritize hard constraints (battery, thermal) before performance.
- Adaptive thresholds: learn switching points from historical data.
- Gradual adaptation: smooth transitions to preserve UX.
- Learning integration: continuously improve adaptation policies.

Quiz 2

Answer 2

In Battery Critical mode, which choice best aligns with the framework?

A) Increase frame rate to reduce motion blur
B) Switch to lightweight motion detection and lower resolution
C) Enable HDR and wide dynamic range
D) Run all detectors in parallel for redundancy

When power is constrained, the system selects energy-saving models and reduces workload.

Scenario

You're designing an adaptive smart camera system that modifies its processing strategy based on environment and resource constraints. The camera supports multiple detection modes (motion, object, face), variable resolution and frame rate, dynamic power management, and adaptive quality settings. It must handle four adaptation scenarios:

1. High Activity Period: many objects, good lighting.
2. Battery Critical: low power mode required.
3. Thermal Throttling: device is hot.
4. Night Mode: low light; simplified detection.

Your solution framework emphasizes:

- Hierarchical decision making: prioritize hard constraints (battery, thermal) before performance.
- Adaptive thresholds: learn switching points from historical data.
- Gradual adaptation: smooth transitions to preserve UX.
- Learning integration: continuously improve adaptation policies.

Quiz 2

Question 3

Why use adaptive thresholds instead of fixed ones for switching modes?

A) Fixed thresholds are always optimal
B) Adaptive thresholds can learn context-specific switch points from history
C) Thresholds are unrelated to adaptation
D) They eliminate the need for monitoring

Scenario

You're designing an adaptive smart camera system that modifies its processing strategy based on environment and resource constraints. The camera supports multiple detection modes (motion, object, face), variable resolution and frame rate, dynamic power management, and adaptive quality settings. It must handle four adaptation scenarios:

1. High Activity Period: many objects, good lighting.
2. Battery Critical: low power mode required.
3. Thermal Throttling: device is hot.
4. Night Mode: low light; simplified detection.

Your solution framework emphasizes:

- Hierarchical decision making: prioritize hard constraints (battery, thermal) before performance.
- Adaptive thresholds: learn switching points from historical data.
- Gradual adaptation: smooth transitions to preserve UX.
- Learning integration: continuously improve adaptation policies.

Quiz 2

Answer 3

Why use adaptive thresholds instead of fixed ones for switching modes?

A) Fixed thresholds are always optimal
B) Adaptive thresholds can learn context-specific switch points from history
C) Thresholds are unrelated to adaptation
D) They eliminate the need for monitoring

Learned thresholds adapt to device, environment, and user patterns.

Scenario

You're designing an adaptive smart camera system that modifies its processing strategy based on environment and resource constraints. The camera supports multiple detection modes (motion, object, face), variable resolution and frame rate, dynamic power management, and adaptive quality settings. It must handle four adaptation scenarios:

1. High Activity Period: many objects, good lighting.
2. Battery Critical: low power mode required.
3. Thermal Throttling: device is hot.
4. Night Mode: low light; simplified detection.

Your solution framework emphasizes:

- Hierarchical decision making: prioritize hard constraints (battery, thermal) before performance.
- Adaptive thresholds: learn switching points from historical data.
- Gradual adaptation: smooth transitions to preserve UX.
- Learning integration: continuously improve adaptation policies.

Quiz 2

Question 4

During Thermal Throttling, which action is most appropriate?

A) Raise GPU clocks temporarily
B) Maintain current load to avoid oscillations
C) Reduce compute intensity and possibly frame rate while cooling
D) Disable temperature sensors

Scenario

You're designing an adaptive smart camera system that modifies its processing strategy based on environment and resource constraints. The camera supports multiple detection modes (motion, object, face), variable resolution and frame rate, dynamic power management, and adaptive quality settings. It must handle four adaptation scenarios:

1. High Activity Period: many objects, good lighting.
2. Battery Critical: low power mode required.
3. Thermal Throttling: device is hot.
4. Night Mode: low light; simplified detection.

Your solution framework emphasizes:

- Hierarchical decision making: prioritize hard constraints (battery, thermal) before performance.
- Adaptive thresholds: learn switching points from historical data.
- Gradual adaptation: smooth transitions to preserve UX.
- Learning integration: continuously improve adaptation policies.

Quiz 2

Answer 4

During Thermal Throttling, which action is most appropriate?

A) Raise GPU clocks temporarily
B) Maintain current load to avoid oscillations
C) Reduce compute intensity and possibly frame rate while cooling
D) Disable temperature sensors

Lowering workload manages heat and prevents throttling spirals.

Scenario

You're designing an adaptive smart camera system that modifies its processing strategy based on environment and resource constraints. The camera supports multiple detection modes (motion, object, face), variable resolution and frame rate, dynamic power management, and adaptive quality settings. It must handle four adaptation scenarios:

1. High Activity Period: many objects, good lighting.
2. Battery Critical: low power mode required.
3. Thermal Throttling: device is hot.
4. Night Mode: low light; simplified detection.

Your solution framework emphasizes:

- Hierarchical decision making: prioritize hard constraints (battery, thermal) before performance.
- Adaptive thresholds: learn switching points from historical data.
- Gradual adaptation: smooth transitions to preserve UX.
- Learning integration: continuously improve adaptation policies.

Quiz 2

Question 5

High Activity with good lighting suggests which strategy?

A) Use higher FPS and object detection with batching, within power/thermal limits
B) Simplify to motion-only detection
C) Turn off detection to preserve battery
D) Switch to Night Mode

Scenario

You're designing an adaptive smart camera system that modifies its processing strategy based on environment and resource constraints. The camera supports multiple detection modes (motion, object, face), variable resolution and frame rate, dynamic power management, and adaptive quality settings. It must handle four adaptation scenarios:

1. High Activity Period: many objects, good lighting.
2. Battery Critical: low power mode required.
3. Thermal Throttling: device is hot.
4. Night Mode: low light; simplified detection.

Your solution framework emphasizes:

- Hierarchical decision making: prioritize hard constraints (battery, thermal) before performance.
- Adaptive thresholds: learn switching points from historical data.
- Gradual adaptation: smooth transitions to preserve UX.
- Learning integration: continuously improve adaptation policies.

Quiz 2

Answer 5

High Activity with good lighting suggests which strategy?

A) Use higher FPS and object detection with batching, within power/thermal limits
B) Simplify to motion-only detection
C) Turn off detection to preserve battery
D) Switch to Night Mode

Good conditions and activity justify richer analytics, constrained by higher-priority limits.

Scenario

You're designing an adaptive smart camera system that modifies its processing strategy based on environment and resource constraints. The camera supports multiple detection modes (motion, object, face), variable resolution and frame rate, dynamic power management, and adaptive quality settings. It must handle four adaptation scenarios:

1. High Activity Period: many objects, good lighting.
2. Battery Critical: low power mode required.
3. Thermal Throttling: device is hot.
4. Night Mode: low light; simplified detection.

Your solution framework emphasizes:

- Hierarchical decision making: prioritize hard constraints (battery, thermal) before performance.
- Adaptive thresholds: learn switching points from historical data.
- Gradual adaptation: smooth transitions to preserve UX.
- Learning integration: continuously improve adaptation policies.

Quiz 2

Question 6

What is the main purpose of gradual adaptation?

A) Faster switching between all modes
B) Avoid collecting telemetry
C) Guarantee maximum accuracy
D) Prevent user-visible disruptions by smoothing transitions

Scenario

You're designing an adaptive smart camera system that modifies its processing strategy based on environment and resource constraints. The camera supports multiple detection modes (motion, object, face), variable resolution and frame rate, dynamic power management, and adaptive quality settings. It must handle four adaptation scenarios:

1. High Activity Period: many objects, good lighting.
2. Battery Critical: low power mode required.
3. Thermal Throttling: device is hot.
4. Night Mode: low light; simplified detection.

Your solution framework emphasizes:

- Hierarchical decision making: prioritize hard constraints (battery, thermal) before performance.
- Adaptive thresholds: learn switching points from historical data.
- Gradual adaptation: smooth transitions to preserve UX.
- Learning integration: continuously improve adaptation policies.

Quiz 2

Answer 6

What is the main purpose of gradual adaptation?

A) Faster switching between all modes
B) Avoid collecting telemetry
C) Guarantee maximum accuracy
D) Prevent user-visible disruptions by smoothing transitions

Ramp-based changes in FPS/quality preserve UX.

Scenario

You're designing an adaptive smart camera system that modifies its processing strategy based on environment and resource constraints. The camera supports multiple detection modes (motion, object, face), variable resolution and frame rate, dynamic power management, and adaptive quality settings. It must handle four adaptation scenarios:

1. High Activity Period: many objects, good lighting.
2. Battery Critical: low power mode required.
3. Thermal Throttling: device is hot.
4. Night Mode: low light; simplified detection.

Your solution framework emphasizes:

- Hierarchical decision making: prioritize hard constraints (battery, thermal) before performance.
- Adaptive thresholds: learn switching points from historical data.
- Gradual adaptation: smooth transitions to preserve UX.
- Learning integration: continuously improve adaptation policies.

Quiz 2

Question 7

Night Mode primarily optimizes for:

A) Visual aesthetics only
B) Low light robustness via simplified or specialized detectors and exposure tweaks
C) Maximum throughput at any cost
D) Thermal stress testing

Scenario

You're designing an adaptive smart camera system that modifies its processing strategy based on environment and resource constraints. The camera supports multiple detection modes (motion, object, face), variable resolution and frame rate, dynamic power management, and adaptive quality settings. It must handle four adaptation scenarios:

1. High Activity Period: many objects, good lighting.
2. Battery Critical: low power mode required.
3. Thermal Throttling: device is hot.
4. Night Mode: low light; simplified detection.

Your solution framework emphasizes:

- Hierarchical decision making: prioritize hard constraints (battery, thermal) before performance.
- Adaptive thresholds: learn switching points from historical data.
- Gradual adaptation: smooth transitions to preserve UX.
- Learning integration: continuously improve adaptation policies.

Quiz 2

Answer 7

Night Mode primarily optimizes for:

A) Visual aesthetics only
B) Low light robustness via simplified or specialized detectors and exposure tweaks
C) Maximum throughput at any cost
D) Thermal stress testing

It tailors detection to low light, often simplifying models and adjusting camera settings.

Scenario

You're designing an adaptive smart camera system that modifies its processing strategy based on environment and resource constraints. The camera supports multiple detection modes (motion, object, face), variable resolution and frame rate, dynamic power management, and adaptive quality settings. It must handle four adaptation scenarios:

1. High Activity Period: many objects, good lighting.
2. Battery Critical: low power mode required.
3. Thermal Throttling: device is hot.
4. Night Mode: low light; simplified detection.

Your solution framework emphasizes:

- Hierarchical decision making: prioritize hard constraints (battery, thermal) before performance.
- Adaptive thresholds: learn switching points from historical data.
- Gradual adaptation: smooth transitions to preserve UX.
- Learning integration: continuously improve adaptation policies.

Quiz 2

Question 8

Which monitoring signal best supports learning integration?

A) Randomly generated labels
B) Only device serial number
C) Number of app installs
D) Latency, temperature, battery drain, detection confidence/accuracy over time

Scenario

You're designing an adaptive smart camera system that modifies its processing strategy based on environment and resource constraints. The camera supports multiple detection modes (motion, object, face), variable resolution and frame rate, dynamic power management, and adaptive quality settings. It must handle four adaptation scenarios:

1. High Activity Period: many objects, good lighting.
2. Battery Critical: low power mode required.
3. Thermal Throttling: device is hot.
4. Night Mode: low light; simplified detection.

Your solution framework emphasizes:

- Hierarchical decision making: prioritize hard constraints (battery, thermal) before performance.
- Adaptive thresholds: learn switching points from historical data.
- Gradual adaptation: smooth transitions to preserve UX.
- Learning integration: continuously improve adaptation policies.

Quiz 2

Answer 8

Which monitoring signal best supports learning integration?

A) Randomly generated labels
B) Only device serial number
C) Number of app installs
D) Latency, temperature, battery drain, detection confidence/accuracy over time

These metrics enable evaluating outcomes and improving policies.

Scenario

You're designing an adaptive smart camera system that modifies its processing strategy based on environment and resource constraints. The camera supports multiple detection modes (motion, object, face), variable resolution and frame rate, dynamic power management, and adaptive quality settings. It must handle four adaptation scenarios:

1. High Activity Period: many objects, good lighting.
2. Battery Critical: low power mode required.
3. Thermal Throttling: device is hot.
4. Night Mode: low light; simplified detection.

Your solution framework emphasizes:

- Hierarchical decision making: prioritize hard constraints (battery, thermal) before performance.
- Adaptive thresholds: learn switching points from historical data.
- Gradual adaptation: smooth transitions to preserve UX.
- Learning integration: continuously improve adaptation policies.

Quiz 2

Question 9

When should model switching criteria favor hysteresis (separate enter/exit thresholds)?

A) Never; single points are enough
B) Only on desktop systems
C) When frequent small fluctuations could cause oscillations between modes
D) Only at device boot

Scenario

You're designing an adaptive smart camera system that modifies its processing strategy based on environment and resource constraints. The camera supports multiple detection modes (motion, object, face), variable resolution and frame rate, dynamic power management, and adaptive quality settings. It must handle four adaptation scenarios:

1. High Activity Period: many objects, good lighting.
2. Battery Critical: low power mode required.
3. Thermal Throttling: device is hot.
4. Night Mode: low light; simplified detection.

Your solution framework emphasizes:

- Hierarchical decision making: prioritize hard constraints (battery, thermal) before performance.
- Adaptive thresholds: learn switching points from historical data.
- Gradual adaptation: smooth transitions to preserve UX.
- Learning integration: continuously improve adaptation policies.

Quiz 2

Answer 9

When should model switching criteria favor hysteresis (separate enter/exit thresholds)?

A) Never; single points are enough
B) Only on desktop systems
C) When frequent small fluctuations could cause oscillations between modes
D) Only at device boot

Hysteresis prevents rapid back-and-forth changes around a boundary.

Scenario

You're designing an adaptive smart camera system that modifies its processing strategy based on environment and resource constraints. The camera supports multiple detection modes (motion, object, face), variable resolution and frame rate, dynamic power management, and adaptive quality settings. It must handle four adaptation scenarios:

1. High Activity Period: many objects, good lighting.
2. Battery Critical: low power mode required.
3. Thermal Throttling: device is hot.
4. Night Mode: low light; simplified detection.

Your solution framework emphasizes:

- Hierarchical decision making: prioritize hard constraints (battery, thermal) before performance.
- Adaptive thresholds: learn switching points from historical data.
- Gradual adaptation: smooth transitions to preserve UX.
- Learning integration: continuously improve adaptation policies.

Quiz 2

Question 10

If historical data shows that at 20% battery users still expect full performance for short sessions, an adaptive policy should:

A) Hard-cap at low-power mode at 30% battery regardless
B) Ignore battery level entirely
C) Learn a context-aware exception allowing temporary high-performance bursts with time/thermal guards
D) Permanently disable learning

Scenario

You're designing an adaptive smart camera system that modifies its processing strategy based on environment and resource constraints. The camera supports multiple detection modes (motion, object, face), variable resolution and frame rate, dynamic power management, and adaptive quality settings. It must handle four adaptation scenarios:

1. High Activity Period: many objects, good lighting.
2. Battery Critical: low power mode required.
3. Thermal Throttling: device is hot.
4. Night Mode: low light; simplified detection.

Your solution framework emphasizes:

- Hierarchical decision making: prioritize hard constraints (battery, thermal) before performance.
- Adaptive thresholds: learn switching points from historical data.
- Gradual adaptation: smooth transitions to preserve UX.
- Learning integration: continuously improve adaptation policies.

Quiz 2

Answer 10

If historical data shows that at 20% battery users still expect full performance for short sessions, an adaptive policy should:

A) Hard-cap at low-power mode at 30% battery regardless
B) Ignore battery level entirely
C) Learn a context-aware exception allowing temporary high-performance bursts with time/thermal guards
D) Permanently disable learning

Adaptive thresholds plus constraints can permit short, safe performance bursts aligned with user behavior.

Scenario

You're designing an adaptive smart camera system that modifies its processing strategy based on environment and resource constraints. The camera supports multiple detection modes (motion, object, face), variable resolution and frame rate, dynamic power management, and adaptive quality settings. It must handle four adaptation scenarios:

1. High Activity Period: many objects, good lighting.
2. Battery Critical: low power mode required.
3. Thermal Throttling: device is hot.
4. Night Mode: low light; simplified detection.

Your solution framework emphasizes:

- Hierarchical decision making: prioritize hard constraints (battery, thermal) before performance.
- Adaptive thresholds: learn switching points from historical data.
- Gradual adaptation: smooth transitions to preserve UX.
- Learning integration: continuously improve adaptation policies.

Learning from Adaptation

Continuous System Improvement

Experience Collection Strategies: Adaptive systems must carefully collect and analyze adaptation experiences to improve future decision making. This includes recording context information, adaptation decisions, and resulting performance outcomes.

Temporal Credit Assignment

- Determining adaptation effectiveness
- Long-term performance tracking
- Delayed effect consideration
- Attribution complexity management

Generalization Challenges

- Cross-condition strategy adaptation
- Condition similarity recognition
- Novel situation handling
- Strategy boundary identification

Meta-Learning Applications: Advanced systems use meta-learning to learn how to adapt more effectively. These systems learn adaptation strategies that generalize across different types of workloads and operating conditions, discovering superior adaptation approaches through experience.

Hierarchical Adaptation

Multi-Level System Intelligence

Time-Scale Separation: Effective adaptive systems operate at multiple time scales with different adaptation mechanisms for immediate responses (microseconds), short-term optimizations (seconds), and long-term strategic planning (minutes to hours).

Adaptation Hierarchy

- **Reactive (µs):** Immediate response to critical events
- **Tactical (ms):** Short-term resource adjustments
- **Strategic (s):** Configuration optimization
- **Planning (min-hr):** Long-term system evolution

Coordination Mechanisms

- Higher-level strategic constraints
- Lower-level feedback information
- Conflict resolution protocols
- Hierarchical decision authority

Emergent Behavior Management: Complex adaptive systems can exhibit emergent behaviors arising from interactions between different adaptation mechanisms. Systems must monitor for and manage unexpected emergent behaviors while maintaining system stability.

SESSION 3: ENERGY-AWARE ML SYSTEMS



Power-Performance Optimization Foundations

Energy Efficiency Imperative: For many embedded and edge systems, optimizing for energy consumption is one of the most critical aspects of ML system design. Battery life often defines the useful lifetime of a device, making energy optimization paramount.

Key Insights - Energy Usage Optimization

- Microcontrollers can run at less than a milliwatt - 1000x less than phone CPUs
- Can run on coin battery or energy harvesting for weeks, months, or years
- Battery capacity typically limited by physical product size
- Main controllable factor: how much energy your system uses

Energy vs Power Concepts

- **Power:** Energy over time (watts)
- **Energy:** Total consumption (joules)
- **Average Power:** Direct lifetime predictor
- **1 mW system:** Lasts 2x longer than 2 mW

Design Trade-offs

- Performance vs energy efficiency
- Accuracy vs power consumption
- Responsiveness vs battery life
- Computational complexity vs energy cost

Component Power Usage Analysis

Understanding Energy Consumption Patterns

Developing Energy Intuition: Most engineers understand operation timing (network requests vs RAM access), but energy thinking requires new mental models. Building intuition about energy magnitudes is essential for power efficiency planning.

Component Type	Power Consumption	Context
Arm Cortex-A9 CPU	500-2,000 mW	Smartphone processor
Display	400 mW	Mobile screen
Active Cell Radio	800 mW	Cellular communication
Bluetooth	100 mW	Standard Bluetooth
Microphone Sensor	300 µW	Audio capture
Bluetooth Low Energy	40 mW	BLE communication
Image Sensor (320x320)	1 mW	30 FPS monochrome
Ambiq Cortex-M4F	1 mW	48 MHz embedded MCU
Accelerometer	1 mW	Motion sensing

Key Insight: Radio communication uses significantly more power than other functionality. The gap between sensor/processor energy and communications power is likely to increase in the future.

Energy Storage and Harvesting



Power Source Capabilities and Limitations

Energy Storage Options: Understanding energy storage capabilities helps determine realistic system lifetimes and operational parameters for different deployment scenarios.

Power Source	Energy Capacity	Lifetime for 1 mW System
CR2032 Coin Battery	2,500 J	~1 month
AA Battery	15,000 J	~6 months

Energy Harvesting Sources

- **Industrial Temperature Differential:** 1-10 mW/cm²
- **Indoor Light:** 10 µW/cm²
- **Outdoor Light:** 10 mW/cm²

Practical Implications

- Only industrial thermal and outdoor solar currently practical
- Energy requirements dropping → more sources viable
- Commercial suppliers: Matrix, e-peas

Planning Guide: These figures help sketch system feasibility for different lifetime, cost, and size requirements, providing initial feasibility checks and design intuitions.

Hardware Selection for Energy Efficiency

Choosing Components Wisely

Component Selection Strategy: When you have a rough idea of required components, you need to evaluate real parts for purchase. Different sources offer varying levels of documentation, support, and optimization for power usage.

Hobbyist-Friendly Sources
Suppliers:
- SparkFun, Arduino, AdaFruit

Advantages:
- Tutorials and drivers included
- Connection advice provided
- Complete prototyping systems

Disadvantages:
- Limited selection
- May not be power-optimized
- Premium pricing

Industrial Sources
Suppliers:
- Digi-Key, Mouser Electronics, Alibaba

Advantages:
- More choice and lower prices
- Detailed datasheets

Challenges:
- Limited support
- Complex power usage documentation.

Datasheet Power Analysis Tip: Search for "milliamps" or "ma" within datasheets - these are often the units used to express power usage. Multiply current by voltage to get power consumption in watts

Measuring Real Power Usage

Validation and Testing Strategies

Practical Power Measurement: Once you have a complete system, testing actual power usage reveals real-world performance. Datasheets and estimates are helpful for planning, but integration testing often shows much higher power consumption than expected.

Measurement Tools

- Multimeter for electrical measurements
- Professional power analyzers
- Built-in current sensing
- Battery lifetime testing

Battery Test Method

- Most reliable approach
- Place known capacity battery
- Measure actual runtime
- Captures all real-world effects
- Includes voltage drop failures
- Simple enough for software engineers!

Why Battery Testing Works Best: This experimental approach captures all effects you care about, including failures when voltage drops too low, which won't show up in simple modeling calculations. Even first attempts running hours/days provide valuable feedback.

Model Power Estimation

Predicting Inference Energy Consumption

Basic Estimation Approach: The simplest way to estimate model power usage is measuring latency for one inference, then multiplying by average system power usage for that time period to get energy consumption.

Practical Example Calculation

- **Model:** Person detector (60 million operations)
- **Chip:** Arm Cortex-M4 at 48 MHz
- **Capability:** 2 x 8-bit multiply/adds per cycle using DSP extensions
- **Estimated Latency:** 30,000,000 / 48,000,000 = 625 ms
- **Power Usage:** 2 mW
- **Energy per Inference:** 1.25 mJ

Early Project Considerations

- Use ballpark figures for feasibility
- Take estimates with "big pinch of salt"
- Consider as upper bound achievement
- Datasheet limitations awareness

Missing Factors

- Memory system overhead
- Peripheral power consumption
- Real-world optimization effects
- System integration inefficiencies

Duty Cycling Optimization

Intelligent Sleep Management

Duty Cycling Fundamentals: Almost all embedded processors can enter sleep mode with very little power consumption while maintaining ability to wake up after intervals or external signals. This enables significant power reduction by inserting sleeps between inference calls.

Sleep Mode Benefits

- Minimal power consumption
- Timer-based wake-up capability
- External signal responsiveness
- Configurable sleep depth
- Rapid wake-up when needed

Continuous Data Collection

- DMA capabilities enable sensor sampling
- ADCs can store results in memory
- No main processor involvement
- Continuous monitoring while sleeping
- Wake on data threshold events

Power-Computation Trade-off: Duty cycling and frequency reduction offer the ability to trade computation for power usage. If you can reduce software latency, you can trade that improvement for lower power budget through increased sleep time or reduced operating frequency.

Dynamic Voltage and Frequency Scaling

Intelligent Power Management

DVFS Principles: Modern processors support dynamic adjustment of operating voltage and frequency. Lower frequencies reduce power consumption but increase execution time, creating complex optimization problems that ML can help solve.

Predictive DVFS

- Traditional: React to current workload
- Intelligent: Predict future requirements
- Proactive frequency adjustment
- Reduced transition latency

Workload Characterization

- Computational requirement analysis
- Historical performance data
- Optimal frequency prediction
- ML-based optimization

Integrated Optimization Factors

- **Thermal Integration:** Proactive frequency scaling to prevent overheating
- **Battery State:** Performance vs lifetime trade-offs based on battery level
- **Cooling Capabilities:** Consider available thermal management

Cascading System Intelligence

Multi-Stage Intelligent Processing

Cascading Design Philosophy: Machine learning's advantage over traditional programming is graceful scaling of compute and storage resources. This enables creating cascades of models with increasing complexity and accuracy at each stage.

Cascade Architecture Benefits

- **Small Model:** Minimal compute, high likelihood triggering, false positives acceptable
- **Medium Model:** Better accuracy, activated when interesting events detected
- **Large Model:** Highest accuracy, used for final verification
- **Energy Efficiency:** Complex models run only when needed

Real-World Example: Voice Interfaces

- DSP continuously monitors microphone
- Small model listens for wake words
- Main CPU sleeps until triggered
- Large model confirms wake word
- Cloud processing for full speech

Design Considerations

- False positive frequency management
- Cloud communication energy cost
- Rare but acceptable radio usage
- Overall energy budget fit

Adaptive Threshold Management



Dynamic Cascade Optimization

Intelligent Cascade Design: Cascading systems use multiple processing stages with increasing computational complexity and accuracy. The challenge is optimizing the entire cascade rather than individual stages to achieve the best overall energy-accuracy trade-off.

Traditional Fixed Thresholds

- Static threshold values
- One-size-fits-all approach
- Suboptimal for varying conditions
- No adaptation capability

Adaptive Threshold Management

- Dynamic threshold adjustment
- Energy availability consideration
- Accuracy requirement adaptation
- Input characteristic responsiveness

Early Exit Strategies: Advanced cascading systems can exit processing early when confidence is sufficiently high, avoiding unnecessary computation. Learning when to exit requires balancing confidence levels with energy savings through intelligent threshold management.

Cross-Stage Optimization: Intelligent cascading considers the entire processing pipeline when making optimization decisions rather than optimizing each stage independently, leading to better overall system performance.

Battery Lifetime Optimization

Long-term Energy Management

Battery Degradation Considerations: Battery capacity decreases over time due to charge cycles, temperature exposure, and aging. Intelligent energy management considers long-term battery health in optimization decisions.

Degradation Factors

- **Charge Cycles:** Each full charge/discharge cycle
- **Temperature:** Heat accelerates chemical degradation
- **Deep Discharge:** Draining battery too low
- **Age:** Calendar-based capacity loss

Optimization Strategies

- **Cycle Management:** Minimize unnecessary cycles
- **Thermal Protection:** Avoid high-temperature operation
- **Conservative Operation:** Trade performance for longevity
- **Predictive Maintenance:** Anticipate replacement needs

Lifetime-Aware Operation: Systems can trade short-term performance for extended operational lifetime by operating more conservatively when battery degradation is detected or predicted. This includes reducing peak power demands and avoiding conditions that accelerate battery aging.



Thermal-Aware Optimization

Managing Heat and Performance

Thermal Management Integration: Thermal optimization becomes particularly important in compact embedded systems where heat dissipation is limited. Intelligent thermal management systems predict temperature evolution and enable proactive thermal control.

Thermal Prediction Models

- Planned workload heat generation
- Environmental condition effects
- Thermal history influence
- Proactive vs reactive management

Thermal-Performance Co-optimization

- Simultaneous optimization approach
- Performance and thermal constraints
- Integrated decision making
- Balanced system operation

Cooling Integration Strategies

- **Passive Cooling:** Heat sink and thermal spreading optimization
- **Active Cooling:** Fan control with energy cost consideration
- **Ambient Adaptation:** Temperature-based performance adjustment
- **Workload Scheduling:** Thermal-aware task distribution



Cross-Scale Energy Optimization

Optimization Across Deployment Scales

Scale-Dependent Strategy Requirements: Energy optimization strategies that work well for microcontrollers may not be appropriate for edge devices or cloud deployments. Systems must adapt their optimization approaches based on deployment scale and available resources.

Deployment Scale	Power Budget	Optimization Focus	Key Strategies
Microcontrollers	Milliwatts	Battery lifetime	Duty cycling, DVFS, sleep modes
Edge Devices	Watts	Thermal management	Dynamic frequency scaling, model switching
Cloud Instances	Kilowatts	Cost efficiency	Resource allocation, workload migration

Coordination Across Scales

Distributed systems spanning multiple deployment scales must coordinate their energy optimization strategies to achieve global optimization objectives rather than local sub-optimization. This includes intelligent workload migration and resource allocation decisions.



Real-time Adaptation Metrics

Monitoring and Control Systems

Adaptive Monitoring Requirements: Energy-aware systems require sophisticated monitoring that adapts its granularity and frequency based on current operating conditions. More frequent monitoring during critical periods, less frequent during stable operation.

Feedback Control Integration

- Control theory principles application
- Desired characteristic maintenance
- Oscillation and instability avoidance
- System behavior stability

Predictive Metrics

- Future performance prediction
- Energy requirement forecasting
- Proactive optimization decisions
- Current condition extension

Multi-Objective Tracking: Systems must simultaneously track multiple metrics including energy consumption, performance, accuracy, and thermal conditions while maintaining lightweight monitoring overhead. The challenge is balancing monitoring completeness with system efficiency.

Mapping Strategies for Different Architectures

Optimization Technique	CNNs	Transformers	MLPs	Rationale
Dataflow Strategy	Weight Stationary	Activation Stationary	Weight Stationary	CNNs reuse filters; Transformers reuse KV-cache; MLPs reuse weights
Memory Layout	NCHW (Channel-Major)	NHWC (Row-Major)	NHWC (Row-Major)	CNNs favor channel-major for convolution; Others prioritize row-major
Kernel Fusion	Convolution + Activation	Fused Attention	GEMM Fusion	Architecture-specific operation combinations
Tiling Strategy	Spatial Tiling	Temporal Tiling	Blocked Tiling	Optimized for dominant computation patterns

Architecture-Specific Optimization

Architecture-Tailored Mapping: While foundational mapping techniques apply broadly, their effectiveness varies based on computational structure, data access patterns, and parallelization opportunities of different neural network architectures.

Key Insight: Each machine learning model benefits from different optimization technique combinations, reinforcing the importance of tailoring execution strategies to workload computational and memory characteristics.

CNN Optimization Strategies

Convolutional Neural Network Mapping

CNN Characteristics: CNNs are characterized by structured spatial computations where small filters (kernels) are repeatedly applied across input feature maps. This structured weight reuse makes weight stationary execution the most effective strategy.

Weight Stationary Benefits

- Filters remain in fast memory
- Activations stream through system
- Maximizes arithmetic intensity
- Minimizes redundant memory transfers
- Weights applied to multiple spatial locations

Channel-Major Layout Advantages

- NCHW format optimization
- Efficient memory coalescing
- Optimized cache locality
- Contiguous memory block fetching
- Reduced latency and improved throughput

CNN-Specific Optimizations

- **Kernel Fusion:** Convolution + ReLU + BatchNorm in single step
- **Spatial Tiling:** Input feature maps processed in smaller subregions
- **Memory Hierarchy:** Keep activations, weights, outputs in fast memory
- **Arithmetic Intensity:** Maximize weight reuse across spatial locations

Transformer Optimization Strategies

Attention Mechanism Optimization

Transformer Characteristics: Unlike CNNs with structured spatial computations, Transformers process variable-length sequences and rely heavily on attention mechanisms. The primary bottleneck is memory bandwidth due to frequent key-value pair access.

Activation Stationary Strategy

- Key-value activations in fast memory
- Query vectors streamed dynamically
- Maximizes activation reuse
- Minimizes redundant memory fetches
- Critical for long-sequence tasks

Row-Major Layout Benefits

- NHWC format for sequence processing
- Contiguous activation access
- Reduced cache misses
- Improved memory coalescing
- Optimized matrix multiplications

Transformer-Specific Optimizations

- **Fused Attention:** Query-key dot products + softmax + weighted sum in single operation
- **Temporal Tiling:** Process sequence blocks efficiently for memory locality
- **KV-Cache Management:** Efficient storage and retrieval of key-value pairs
- **Memory Traffic Reduction:** Eliminate intermediate writes in attention computation

MLP Optimization Strategies

Multi-Layer Perceptron Mapping

MLP Characteristics: MLPs primarily consist of fully connected layers where large matrices of weights and activations are multiplied. Weight stationary execution is most effective, similar to CNNs, due to weight reuse across multiple activations in a batch.

Weight Stationary Implementation

- Weights kept in local memory
- Activations streamed dynamically
- Weight matrices reused across batches
- Reduced weight reload frequency
- Optimized for dense layer computation

GEMM Fusion Benefits

- General Matrix Multiplication optimization
- Dense layer + activation fusion
- Bias addition integration
- Single optimized kernel execution
- Reduced memory traffic

MLP-Specific Optimizations

- **Blocked Tiling:** Large matrix multiplications divided into memory-friendly sub-blocks
- **Row-Major Layout:** NHWC format for efficient batch processing
- **Shared Memory Utilization:** Frequently accessed matrix portions in fast memory
- **Bandwidth Optimization:** Minimize external memory accesses through intelligent blocking

Hybrid Mapping Strategies

Layer-Specific Optimization

Hybrid Approach Necessity: Real-world architectures often involve diverse computational requirements that cannot be effectively addressed with a single, fixed approach. Hybrid mapping strategies dynamically apply different optimizations to specific layers.

Mixed Architecture Examples

- **Vision Transformers:** Convolution + attention
- **ResNets:** Convolution + skip connections
- **Hybrid CNNs:** Spatial + fully connected layers
- **Mobile Networks:** Depthwise + pointwise convolutions

Layer-Specific Optimization

- **Convolutional Layers:** Weight stationary
- **Attention Layers:** Activation stationary
- **Dense Layers:** Output stationary
- **Activation Functions:** Kernel fusion

Hardware Implementation Examples

- **Google TPUs:** Weight stationary for convolutions, activation stationary for attention
- **NVIDIA GPUs:** Fused kernels with hybrid memory layouts for different layers
- **Graphcore IPUs:** Dynamic per-layer execution strategy selection

Model Architecture Impact on Latency

Optimizing Latency

Architecture-Level Optimization Priority: After switching hardware platforms, the easiest place to have big impact on neural network latency is at the architecture level. Creating a model that is accurate enough but involves fewer calculations can speed up inference without any code changes.

Model Architecture Benefits

- **Trade-off Flexibility:** Reduced accuracy for increased speed
- **Easy Replacement:** Models are functional black boxes
- **Graceful Degradation:** Neural networks degrade smoothly
- **Layer Removal:** Experiment with removing individual layers

Algorithm vs Code Optimization

- Better to eliminate work entirely
- High-level algorithm changes preferred
- Model swapping easier than code rewriting
- Focus on model architectures first

Training Data Impact

- Start with most accurate model possible
- More scope for accuracy-speed trade-offs
- Training data improvement helps optimization
- Better data enables more aggressive optimization

Week 7 Key Takeaways

Core Optimization Principles:

• ML-driven optimization: Use machine learning to optimize system behavior beyond static heuristics

• Adaptive runtime systems: Enable continuous adaptation to changing conditions and requirements

• Energy-aware design: Balance performance with energy efficiency across deployment scales

• Cross-scale intelligence: Apply optimization principles from microcontrollers to cloud systems

System Design Methodologies:

• Predictive optimization: Learn system behavior patterns to enable proactive optimization decisions

• Hierarchical adaptation: Coordinate optimization across multiple time scales and system levels

• Experience-driven improvement: Continuously learn from adaptation results to improve future decisions

• Multi-objective balancing: Simultaneously optimize latency, accuracy, energy, and other constraints

Practical Implementation Strategies:

• Performance prediction models enable intelligent resource allocation and scheduling decisions

• Adaptive cascading reduces energy consumption while maintaining functionality requirements

• Thermal and battery management extend operational lifetime in resource-constrained environments

• Cross-platform optimization transfers knowledge between different deployment scenarios

Looking Ahead - Week 8 Preview

Automating ML Compilation

Next Week's Focus:

- Compilation pipelines for different deployment targets
- Automated optimization for resource-constrained devices\
- Cross-platform compilation challenges and solutions
- Integration with runtime adaptation systems

Connection to ML for Systems:

- Compilation systems benefit from learned optimization strategies
- Runtime adaptation requires compiler support for dynamic optimization
- Cross-platform deployment leverages system intelligence for target-specific optimization
- Automated toolchains enable broader deployment of intelligent systems

Required Readings for Week 8:

- Book 1 – Volume 1: ML Frameworks
- Book 1 – Volume 1: Hardware Acceleration
- Book 1 – Volume 1: Model Compression
- Book 1 – Volume 1: Model Serving
- Book 2: Chapter 13: TensorFlow Lite for Microcontrollers
- Book 2: Chapter 19: Porting Models from TensorFlow to TensorFlow Lite

